

Chapter 21 — Tree BFS

0. What You Will Learn in This Chapter

Tree BFS is one of the most important traversal patterns for binary trees and N-ary trees.

BFS means **Breadth-First Search**.

In trees, BFS usually means visiting nodes **level by level**.

It is also called **level-order traversal**.

By the end of this chapter, you should understand:

- What Tree BFS is
- How BFS differs from DFS
- Why queue is used
- How to process a tree level by level
- How to track level size
- How to compute depth using BFS
- How to solve right side view
- How to solve zigzag traversal
- How to connect next right pointers
- How to solve vertical order-style problems
- When BFS is better than DFS
- When DFS is better than BFS
- Common queue mistakes
- Common level-size mistakes

Core problems covered in this chapter:

1. Binary Tree Level Order Traversal
 2. Binary Tree Level Order Traversal II
 3. Binary Tree Zigzag Level Order Traversal
 4. Average of Levels in Binary Tree
 5. Maximum Depth of Binary Tree, BFS
 6. Minimum Depth of Binary Tree, BFS
 7. Binary Tree Right Side View
 8. Binary Tree Left Side View
 9. Find Largest Value in Each Tree Row
 10. Populating Next Right Pointers in Each Node
 11. Populating Next Right Pointers in Each Node II
 12. N-ary Tree Level Order Traversal
 13. Cousins in Binary Tree
 14. Check Completeness of a Binary Tree
 15. Vertical Order Traversal of a Binary Tree
 16. Deepest Leaves Sum
 17. Even Odd Tree
 18. Reverse Odd Levels of Binary Tree
-

1. What Is Tree BFS?

Tree BFS visits nodes level by level.

Example tree:

```
  1
 / \
2   3
 / \ \
4  5 6
```

BFS order:

```
1
2, 3
4, 5, 6
```

As a flat traversal:

```
1, 2, 3, 4, 5, 6
```

BFS uses a queue because nodes should be processed in the order they are discovered.

2. Why Tree BFS Exists

DFS goes deep first.

BFS goes wide first.

Some problems require level information:

- nodes level by level
- average per level
- rightmost node at each level
- shortest root-to-leaf depth
- zigzag by level
- largest value per row
- connect nodes horizontally

These are naturally BFS problems.

3. Core Need / Why Use Tree BFS Over DFS?

3.1 Why Not DFS?

DFS can solve many level problems too, but BFS is often simpler when the problem is explicitly level-based.

Example:

Average of levels:

BFS naturally processes one level at a time.

DFS would need depth-indexed arrays.

3.2 Why Queue?

BFS processes first discovered nodes first.

A queue is FIFO:

First In, First Out

This matches level order.

3.3 Why Track Level Size?

At the start of each level, the queue contains exactly the nodes of that level.

Store:

```
int levelSize = q.size();
```

Then process exactly that many nodes.

This prevents mixing next level nodes with current level nodes.

4. When to Use Tree BFS

Use Tree BFS when the problem asks for:

4.1 Level Order

Examples:

- Binary Tree Level Order Traversal
- N-ary Tree Level Order Traversal

4.2 Per-Level Statistics

Examples:

- Average of Levels
- Largest Value in Each Tree Row
- Deepest Leaves Sum

4.3 First / Last Node of Each Level

Examples:

- Right Side View
- Left Side View

4.4 Minimum Depth

BFS is ideal because first leaf found is the nearest leaf.

Example:

- Minimum Depth of Binary Tree

4.5 Zigzag or Alternating Level Behavior

Examples:

- Zigzag Level Order Traversal
- Reverse Odd Levels
- Even Odd Tree

4.6 Horizontal Connections

Examples:

- Populating Next Right Pointers
-

5. When NOT to Use Tree BFS

5.1 Need Subtree Return Values

Use DFS.

Examples:

- Diameter of Binary Tree
- Balanced Binary Tree
- Maximum Path Sum

5.2 Need Root-to-Leaf Path Backtracking

DFS is usually simpler.

Examples:

- Path Sum II
- Binary Tree Paths

5.3 Need Inorder Traversal

Use DFS.

Especially for BST sorted order.

5.4 Tree Is Very Wide

BFS space can be large because it stores entire level.

DFS stores height.

6. Core Intuition

BFS processes tree like waves.

Start from root.

Then visit all nodes distance 1 from root.

Then distance 2.

Then distance 3.

Each loop iteration handles one level.

7. Mental Models

7.1 Queue of Work

Queue stores nodes waiting to be processed.

Pop front node, push its children.

7.2 Fire Spreading

Root is fire source.

Fire spreads to children level by level.

7.3 Level Snapshot

At start of each level:

```
int levelSize = q.size();
```

This is a snapshot of current level.

Process exactly levelSize nodes.

7.4 First Leaf in BFS Is Minimum Depth

Because BFS visits by increasing depth, the first leaf encountered is closest to root.

8. Pattern Recognition Checklist

Ask:

19. Does the problem mention level order?
20. Does it ask for each row/level?
21. Does it need average/largest/sum per level?
22. Does it ask for right side or left side view?
23. Does it ask for minimum depth?

- 24. Does it require zigzag level order?
- 25. Does it connect nodes horizontally?
- 26. Do I need first/last node per level?
- 27. Is shortest root-to-leaf path needed?
- 28. Would DFS require extra depth bookkeeping?

If yes, Tree BFS is likely useful.

9. C++ Tools Needed

9.1 Queue

```
queue<TreeNode*> q;
```

Operations:

```
q.push(node);
q.front();
q.pop();
q.empty();
q.size();
```

Important:

pop() does not return value.

Correct:

```
TreeNode* node = q.front();
q.pop();
```

9.2 Level Size

```
int levelSize = q.size();
```

Use before the inner loop.

9.3 Result Vector

```
vector<vector<int>> result;
```

For level order traversal.

10. Core Templates

Template 1 — Basic Level Order

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> result;

    if (root == nullptr) {
        return result;
    }

    queue<TreeNode*> q;
```

```

q.push(root);

while (!q.empty()) {
    int levelSize = q.size();
    vector<int> level;

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        level.push_back(node->val);

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }

    result.push_back(level);
}

return result;
}

```

Template 2 — BFS With Depth Counter

```

int depth = 0;

queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {
    int levelSize = q.size();
    depth++;

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        if (node->left != nullptr) {

```

```

        q.push(node->left);
    }

    if (node->right != nullptr) {
        q.push(node->right);
    }
}
}
}

```

Template 3 — Minimum Depth BFS

```

int minDepth(TreeNode* root) {
    if (root == nullptr) {
        return 0;
    }

    queue<TreeNode*> q;
    q.push(root);

    int depth = 1;

    while (!q.empty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front();
            q.pop();

            if (node->left == nullptr && node->right == nullptr) {
                return depth;
            }

            if (node->left != nullptr) {
                q.push(node->left);
            }

            if (node->right != nullptr) {
                q.push(node->right);
            }
        }

        depth++;
    }
}

```

```
    return depth;
}
```

Template 4 — Right Side View

```
vector<int> rightSideView(TreeNode* root) {
    vector<int> result;

    if (root == nullptr) {
        return result;
    }

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front();
            q.pop();

            if (i == levelSize - 1) {
                result.push_back(node->val);
            }

            if (node->left != nullptr) {
                q.push(node->left);
            }

            if (node->right != nullptr) {
                q.push(node->right);
            }
        }
    }

    return result;
}
```

Template 5 — Zigzag Level Order

```

bool leftToRight = true;

while (!q.empty()) {
    int levelSize = q.size();
    vector<int> level(levelSize);

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        int index = leftToRight ? i : levelSize - 1 - i;
        level[index] = node->val;

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }

    result.push_back(level);
    leftToRight = !leftToRight;
}

```

11. Major Variations of Tree BFS

Basic level order	process level size	Level Order
Bottom-up level order	reverse result	Level Order II
Zigzag	alternate write direction	Zigzag Traversal
Per-level average	sum / level size	Average Levels
Per-level max	track max	Largest Row Value
Depth count	increment per level	Max Depth
First leaf	return immediately	Min Depth
Side view	first/last per level	Right Side View

Variation	Core Idea	Example
Connect next pointers	link nodes within level	Populating Next
Completeness check	null behavior	Complete Tree
Vertical traversal	row/col ordering	Vertical Traversal

12. Full Problem Walkthrough 1 — LeetCode 102: Binary Tree Level Order Traversal

12.1 Problem Summary

Return level-order traversal of binary tree.

Example:

```

  3
 / \
9  20
 / \
15 7

```

Output:

```
[[3], [9,20], [15,7]]
```

12.2 How to Identify This Pattern

Problem directly asks for level-order traversal.

Use BFS with queue and level size.

12.3 C++ Code

```

class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> result;

        if (root == nullptr) {
            return result;
        }

        queue<TreeNode*> q;
        q.push(root);

```

```

while (!q.empty()) {
    int levelSize = q.size();
    vector<int> level;

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        level.push_back(node->val);

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }

    result.push_back(level);
}

return result;
}
};

```

12.4 Dry Run

Queue starts:

[3]

Level 1:

[3]

Push:

9, 20

Level 2:

[9, 20]

Push:

15, 7

Level 3:

[15, 7]

12.5 Complexity

Time:

$O(n)$

Space:

$O(w)$

where w is maximum width of tree.

Worst case:

$O(n)$

12.6 Common Mistakes

- Forgetting empty root check
 - Not using levelSize
 - Mixing nodes from different levels
 - Calling `q.front()` after `q.pop()`
-

13. Full Problem Walkthrough 2 — LeetCode 107: Binary Tree Level Order Traversal II

13.1 Problem Summary

Return bottom-up level order traversal.

Example output:

[[15,7], [9,20], [3]]

13.2 How to Identify This Pattern

Same as level order, but result should be reversed.

13.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> levelOrderBottom(TreeNode* root) {
        vector<vector<int>> result;

        if (root == nullptr) {
            return result;
        }
    }
}
```

```
queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {
    int levelSize = q.size();
    vector<int> level;

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        level.push_back(node->val);

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }

    result.push_back(level);
}

reverse(result.begin(), result.end());

return result;
}
};
```

13.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

including output.

14. Full Problem Walkthrough 3 — LeetCode 103: Binary Tree Zigzag Level Order Traversal

14.1 Problem Summary

Return zigzag level order traversal.

Example:

Level 0: left to right

Level 1: right to left

Level 2: left to right

14.2 How to Identify This Pattern

Problem asks for level order with alternating direction.

Use BFS and fill level vector by index.

14.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> result;

        if (root == nullptr) {
            return result;
        }

        queue<TreeNode*> q;
        q.push(root);

        bool leftToRight = true;

        while (!q.empty()) {
            int levelSize = q.size();
            vector<int> level(levelSize);

            for (int i = 0; i < levelSize; i++) {
                TreeNode* node = q.front();
                q.pop();

                int index = leftToRight ? i : levelSize - 1 - i;
```

```

        level[index] = node->val;

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }

    result.push_back(level);
    leftToRight = !leftToRight;
}

return result;
}
};

```

14.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

excluding output.

14.5 Common Mistakes

- Reversing queue child insertion order unnecessarily
- Forgetting to toggle direction
- Reversing every level after building, which is okay but less direct

15. Full Problem Walkthrough 4 — LeetCode 637: Average of Levels in Binary Tree

15.1 Problem Summary

Return average value of nodes at each level.

15.2 How to Identify This Pattern

Need per-level average.

BFS naturally processes one level at a time.

15.3 C++ Code

```
class Solution {
public:
    vector<double> averageOfLevels(TreeNode* root) {
        vector<double> result;

        if (root == nullptr) {
            return result;
        }

        queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();
            long long sum = 0;

            for (int i = 0; i < levelSize; i++) {
                TreeNode* node = q.front();
                q.pop();

                sum += node->val;

                if (node->left != nullptr) {
                    q.push(node->left);
                }

                if (node->right != nullptr) {
                    q.push(node->right);
                }
            }

            result.push_back((double)sum / levelSize);
        }

        return result;
    }
};
```

15.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

15.5 Common Mistake

Using int sum may overflow.

Use long long.

16. Full Problem Walkthrough 5 — LeetCode 104: Maximum Depth of Binary Tree, BFS

16.1 Problem Summary

Return maximum depth.

16.2 How to Identify BFS Option

Maximum depth can be solved by DFS or BFS.

BFS counts levels.

16.3 C++ Code

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        queue<TreeNode*> q;
        q.push(root);

        int depth = 0;

        while (!q.empty()) {
            int levelSize = q.size();
            depth++;

            for (int i = 0; i < levelSize; i++) {
                TreeNode* node = q.front();
```

```
        q.pop();

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}

return depth;
}
};
```

16.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

17. Full Problem Walkthrough 6 — LeetCode 111: Minimum Depth of Binary Tree, BFS

17.1 Problem Summary

Return minimum root-to-leaf depth.

17.2 Why BFS Is Excellent Here

BFS visits nodes by increasing depth.

The first leaf encountered gives minimum depth.

17.3 C++ Code

```
class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }
    }
};
```

```

    }

    queue<TreeNode*> q;
    q.push(root);

    int depth = 1;

    while (!q.empty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front();
            q.pop();

            if (node->left == nullptr && node->right == nullptr) {
                return depth;
            }

            if (node->left != nullptr) {
                q.push(node->left);
            }

            if (node->right != nullptr) {
                q.push(node->right);
            }
        }

        depth++;
    }

    return depth;
}
};

```

17.4 Complexity

Time:

$O(n)$

Worst case.

Best case can be much faster if a shallow leaf exists.

Space:

$O(w)$

18. Full Problem Walkthrough 7 — LeetCode 199: Binary Tree Right Side View

18.1 Problem Summary

Return nodes visible from right side.

Example:

```
    1
   /\
  2 3
   /\
  5 4
```

Output:

```
[1,3,4]
```

18.2 How to Identify This Pattern

Need rightmost node of each level.

Use BFS and take last node at each level.

18.3 C++ Code

```
class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        vector<int> result;

        if (root == nullptr) {
            return result;
        }

        queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();

            for (int i = 0; i < levelSize; i++) {
                TreeNode* node = q.front();
                q.pop();
```

```

        if (i == levelSize - 1) {
            result.push_back(node->val);
        }

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}

return result;
}
};

```

18.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

18.5 Similar Problem

Left side view:

take $i == 0$.

19. Full Problem Walkthrough 8 — Left Side View

19.1 Problem Summary

Return nodes visible from left side.

This is not always a LeetCode main problem, but it is a common interview variation.

19.2 C++ Code

```

vector<int> leftSideView(TreeNode* root) {
    vector<int> result;

```

```

if (root == nullptr) {
    return result;
}

queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        if (i == 0) {
            result.push_back(node->val);
        }

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}

return result;
}

```

20. Full Problem Walkthrough 9 — LeetCode 515: Find Largest Value in Each Tree Row

20.1 Problem Summary

Return largest value in each row of binary tree.

20.2 How to Identify This Pattern

Need max per level.

Use BFS with level size.

20.3 C++ Code

```
class Solution {
public:
    vector<int> largestValues(TreeNode* root) {
        vector<int> result;

        if (root == nullptr) {
            return result;
        }

        queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();
            int best = INT_MIN;

            for (int i = 0; i < levelSize; i++) {
                TreeNode* node = q.front();
                q.pop();

                best = max(best, node->val);

                if (node->left != nullptr) {
                    q.push(node->left);
                }

                if (node->right != nullptr) {
                    q.push(node->right);
                }
            }

            result.push_back(best);
        }

        return result;
    }
};
```

20.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

21. Full Problem Walkthrough 10 — LeetCode 116: Populating Next Right Pointers in Each Node

21.1 Problem Summary

Given perfect binary tree, connect each node's next pointer to its right neighbor.
If no neighbor, connect to nullptr.

21.2 How to Identify This Pattern

Need connect nodes within same level.
BFS naturally processes level by level.

21.3 C++ Code, BFS Version

```
class Solution {
public:
    Node* connect(Node* root) {
        if (root == nullptr) {
            return nullptr;
        }

        queue<Node*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();
            Node* prev = nullptr;

            for (int i = 0; i < levelSize; i++) {
                Node* node = q.front();
                q.pop();

                if (prev != nullptr) {
                    prev->next = node;
                }
            }
        }
    }
};
```

```

        prev = node;

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }

    prev->next = nullptr;
}

return root;
}
};

```

21.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

Perfect-tree optimized solution can use $O(1)$ extra space.

22. Full Problem Walkthrough 11 — LeetCode 117: Populating Next Right Pointers in Each Node II

22.1 Problem Summary

Same as previous problem, but binary tree is not necessarily perfect.

22.2 BFS Version

Same BFS level connection works for any binary tree.

22.3 C++ Code

```

class Solution {
public:

```

```

Node* connect(Node* root) {
    if (root == nullptr) {
        return nullptr;
    }

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        int levelSize = q.size();
        Node* prev = nullptr;

        for (int i = 0; i < levelSize; i++) {
            Node* node = q.front();
            q.pop();

            if (prev != nullptr) {
                prev->next = node;
            }

            prev = node;

            if (node->left != nullptr) {
                q.push(node->left);
            }

            if (node->right != nullptr) {
                q.push(node->right);
            }
        }

        prev->next = nullptr;
    }

    return root;
};

```

22.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

23. Full Problem Walkthrough 12 — LeetCode 429: N-ary Tree Level Order Traversal

23.1 Problem Summary

Given N-ary tree, return level order traversal.
Each node has list of children.

23.2 How to Identify This Pattern

Same as binary tree BFS, but push all children.

23.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> levelOrder(Node* root) {
        vector<vector<int>> result;

        if (root == nullptr) {
            return result;
        }

        queue<Node*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();
            vector<int> level;

            for (int i = 0; i < levelSize; i++) {
                Node* node = q.front();
                q.pop();

                level.push_back(node->val);

                for (Node* child : node->children) {
                    if (child != nullptr) {
                        q.push(child);
                    }
                }
            }
        }

        return result;
    }
};
```

```

        }
    }

    result.push_back(level);
}

return result;
}
};

```

23.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

24. Full Problem Walkthrough 13 — LeetCode 993: Cousins in Binary Tree

24.1 Problem Summary

Two nodes are cousins if:

- they have same depth
- they have different parents

Return true if values x and y are cousins.

24.2 How to Identify BFS

Need compare nodes at same level.

BFS is natural.

24.3 C++ Code

```

class Solution {
public:
    bool isCousins(TreeNode* root, int x, int y) {
        if (root == nullptr) {
            return false;
        }

        queue<pair<TreeNode*, TreeNode*>> q;

```

```

q.push({root, nullptr});

while (!q.empty()) {
    int levelSize = q.size();

    TreeNode* parentX = nullptr;
    TreeNode* parentY = nullptr;

    for (int i = 0; i < levelSize; i++) {
        auto [node, parent] = q.front();
        q.pop();

        if (node->val == x) {
            parentX = parent;
        }

        if (node->val == y) {
            parentY = parent;
        }

        if (node->left != nullptr) {
            q.push({node->left, node});
        }

        if (node->right != nullptr) {
            q.push({node->right, node});
        }
    }

    if (parentX != nullptr || parentY != nullptr) {
        return parentX != nullptr &&
            parentY != nullptr &&
            parentX != parentY;
    }
}

return false;
};

```

24.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

25. Full Problem Walkthrough 14 — LeetCode 958: Check Completeness of a Binary Tree

25.1 Problem Summary

A binary tree is complete if every level except possibly last is full, and last level nodes are as far left as possible.

25.2 How to Identify BFS

Completeness is naturally checked by level order.

Once a null child is seen, no later non-null node should appear.

25.3 C++ Code

```
class Solution {
public:
    bool isCompleteTree(TreeNode* root) {
        queue<TreeNode*> q;
        q.push(root);

        bool seenNull = false;

        while (!q.empty()) {
            TreeNode* node = q.front();
            q.pop();

            if (node == nullptr) {
                seenNull = true;
            } else {
                if (seenNull) {
                    return false;
                }

                q.push(node->left);
                q.push(node->right);
            }
        }
    }
};
```

```
    }  
  
    return true;  
}  
};
```

25.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

25.5 Common Mistake

Only pushing non-null children.

For completeness, you must observe null positions, so this solution pushes nulls too.

26. Full Problem Walkthrough 15 — LeetCode 987: Vertical Order Traversal of a Binary Tree

26.1 Problem Summary

Return vertical traversal of binary tree.

Each node has:

- row
- column

Root is (0, 0).

Left child:

row + 1, col - 1

Right child:

row + 1, col + 1

Sort by:

29. column ascending
30. row ascending
31. value ascending if same row and column

26.2 How to Identify This Pattern

Need coordinate traversal and sorted grouping.

Can use DFS or BFS to collect (col, row, value).

Then sort.

26.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> verticalTraversal(TreeNode* root) {
        vector<tuple<int, int, int>> nodes;

        queue<tuple<TreeNode*, int, int>> q;
        q.push({root, 0, 0});

        while (!q.empty()) {
            auto [node, row, col] = q.front();
            q.pop();

            nodes.push_back({col, row, node->val});

            if (node->left != nullptr) {
                q.push({node->left, row + 1, col - 1});
            }

            if (node->right != nullptr) {
                q.push({node->right, row + 1, col + 1});
            }
        }

        sort(nodes.begin(), nodes.end());

        vector<vector<int>> result;

        int i = 0;

        while (i < nodes.size()) {
            int currentCol = get<0>(nodes[i]);
            vector<int> column;

            while (i < nodes.size() && get<0>(nodes[i]) == currentCol) {
                column.push_back(get<2>(nodes[i]));
                i++;
            }

            result.push_back(column);
        }
    }
};
```

```
        return result;
    }
};
```

26.4 Complexity

Time:

$O(n \log n)$

because of sorting.

Space:

$O(n)$

27. Full Problem Walkthrough 16 — LeetCode 1302: Deepest Leaves Sum

27.1 Problem Summary

Return sum of values of deepest leaves.

27.2 How to Identify BFS

Need deepest level.

BFS level order can overwrite sum each level.

Final sum is deepest level sum.

27.3 C++ Code

```
class Solution {
public:
    int deepestLeavesSum(TreeNode* root) {
        queue<TreeNode*> q;
        q.push(root);

        int sum = 0;

        while (!q.empty()) {
            int levelSize = q.size();
            sum = 0;

            for (int i = 0; i < levelSize; i++) {
                TreeNode* node = q.front();
                q.pop();
```

```
        sum += node->val;

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}

return sum;
}
```

27.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

28. Full Problem Walkthrough 17 — LeetCode 1609: Even Odd Tree

28.1 Problem Summary

A binary tree is Even-Odd if:

For even-indexed levels:

- values are odd
- strictly increasing left to right

For odd-indexed levels:

- values are even
- strictly decreasing left to right

Root is level 0.

28.2 How to Identify BFS

Rules depend on level index and left-to-right order.

Use BFS.

28.3 C++ Code

```

class Solution {
public:
    bool isEvenOddTree(TreeNode* root) {
        queue<TreeNode*> q;
        q.push(root);

        int level = 0;

        while (!q.empty()) {
            int levelSize = q.size();

            int prev = level % 2 == 0 ? INT_MIN : INT_MAX;

            for (int i = 0; i < levelSize; i++) {
                TreeNode* node = q.front();
                q.pop();

                int value = node->val;

                if (level % 2 == 0) {
                    if (value % 2 == 0 || value <= prev) {
                        return false;
                    }
                } else {
                    if (value % 2 == 1 || value >= prev) {
                        return false;
                    }
                }

                prev = value;

                if (node->left != nullptr) {
                    q.push(node->left);
                }

                if (node->right != nullptr) {
                    q.push(node->right);
                }
            }

            level++;
        }
    }
}

```

```
        return true;
    }
};
```

28.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

29. Full Problem Walkthrough 18 — LeetCode 2415: Reverse Odd Levels of Binary Tree

29.1 Problem Summary

Given perfect binary tree, reverse node values at each odd level.

29.2 How to Identify BFS

Need operate by level.

At odd levels, reverse values.

29.3 C++ Code

```
class Solution {
public:
    TreeNode* reverseOddLevels(TreeNode* root) {
        if (root == nullptr) {
            return nullptr;
        }

        queue<TreeNode*> q;
        q.push(root);

        int level = 0;

        while (!q.empty()) {
            int levelSize = q.size();
            vector<TreeNode*> nodes;
```

```

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        nodes.push_back(node);

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }

    if (level % 2 == 1) {
        int left = 0;
        int right = nodes.size() - 1;

        while (left < right) {
            swap(nodes[left]->val, nodes[right]->val);
            left++;
            right--;
        }
    }

    level++;
}

return root;
}
};

```

29.4 Complexity

Time:

$O(n)$

Space:

$O(w)$

30. Mini Problem 1 — Level Sum

```

vector<int> levelSums(TreeNode* root) {
    vector<int> result;

    if (root == nullptr) {
        return result;
    }

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int levelSize = q.size();
        int sum = 0;

        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front();
            q.pop();

            sum += node->val;

            if (node->left != nullptr) {
                q.push(node->left);
            }

            if (node->right != nullptr) {
                q.push(node->right);
            }
        }

        result.push_back(sum);
    }

    return result;
}

```

31. Mini Problem 2 — Count Nodes Per Level

```

vector<int> countPerLevel(TreeNode* root) {
    vector<int> result;

```

```

if (root == nullptr) {
    return result;
}

queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {
    int levelSize = q.size();
    result.push_back(levelSize);

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}

return result;
}

```

32. Mini Problem 3 — First Value at Each Level

```

vector<int> firstValueEachLevel(TreeNode* root) {
    vector<int> result;

    if (root == nullptr) {
        return result;
    }

    queue<TreeNode*> q;
    q.push(root);
}

```

```

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        if (i == 0) {
            result.push_back(node->val);
        }

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}

return result;
}

```

33. Common Edge Cases for Tree BFS

33.1 Empty Tree

Return empty result or 0 depending on problem.

33.2 Single Node

One level only.

33.3 Skewed Tree

Queue size remains small, but levels are many.

33.4 Very Wide Tree

Queue may store many nodes.

Space can be $O(n)$.

33.5 Null Nodes

Most BFS problems do not push null nodes.

Completeness check is an exception.

33.6 Level Size Snapshot

Always store level size before processing level.

Do not use changing `q.size()` inside loop condition for level processing.

34. Common Mistakes and Wrong Code

Mistake 1 — Not Taking Level Size Snapshot

Wrong:

```
for (int i = 0; i < q.size(); i++)
```

`q.size()` changes as children are pushed.

Correct:

```
int levelSize = q.size();
```

```
for (int i = 0; i < levelSize; i++)
```

Mistake 2 — Calling `front()` After `pop()`

Wrong:

```
q.pop();  
TreeNode* node = q.front();
```

Correct:

```
TreeNode* node = q.front();  
q.pop();
```

Mistake 3 — Forgetting Null Root Check

Wrong:

```
q.push(root);
```

when root might be null.

Mistake 4 — Mixing Current and Next Level

If `levelSize` is not used, nodes from next level may enter current level processing.

Mistake 5 — Pushing Null Children Unnecessarily

Most problems should only push non-null children.

Completeness check is special.

35. Interview Explanation Templates

35.1 Level Order

I use a queue for BFS. At the beginning of each level, I record the current queue size. Then I process exactly that many nodes, which represents one level.

35.2 Right Side View

During BFS, the last node processed at each level is the visible node from the right side, so I add it to the answer.

35.3 Minimum Depth

BFS visits nodes in increasing depth order. Therefore, the first leaf node encountered gives the minimum depth.

35.4 Zigzag Traversal

I still process nodes level by level with BFS, but I fill the level array from left to right or right to left depending on the level parity.

35.5 Completeness Check

I perform level-order traversal including null positions. Once a null position is seen, every later node must also be null; otherwise the tree is not complete.

36. Complexity Patterns

Level order traversal	$O(n)$	$O(w)$
Bottom-up level order	$O(n)$	$O(n)$
Zigzag traversal	$O(n)$	$O(w)$
Average of levels	$O(n)$	$O(w)$
Max depth BFS	$O(n)$	$O(w)$
Min depth BFS	$O(n)$ worst	$O(w)$
Right side view	$O(n)$	$O(w)$
Largest value per row	$O(n)$	$O(w)$
Connect next pointers BFS	$O(n)$	$O(w)$
Completeness check	$O(n)$	$O(w)$
Vertical traversal	$O(n \log n)$	$O(n)$
Deepest leaves sum	$O(n)$	$O(w)$
Even Odd Tree	$O(n)$	$O(w)$

w = maximum width of tree.

37. Time-Boxed Practice Guidance

Easy Tree BFS

Target:

10–20 minutes

Examples:

- Level Order Traversal
- Average of Levels
- Maximum Depth BFS
- Deepest Leaves Sum

Medium Tree BFS

Target:

20–40 minutes

Examples:

- Zigzag Level Order
- Right Side View
- Minimum Depth
- Cousins in Binary Tree
- Check Completeness

Harder Tree BFS

Target:

40–60 minutes

Examples:

- Vertical Order Traversal
- Populating Next Right Pointers II
- Even Odd Tree
- Reverse Odd Levels

38. Expanded Practice Set

38.1 Must-Do Easy

32. Binary Tree Level Order Traversal
33. Binary Tree Level Order Traversal II
34. Average of Levels in Binary Tree
35. Maximum Depth of Binary Tree, BFS
36. Minimum Depth of Binary Tree, BFS
37. Find Largest Value in Each Tree Row
38. Deepest Leaves Sum

38.2 Must-Do Medium

39. Binary Tree Zigzag Level Order Traversal
40. Binary Tree Right Side View
41. Binary Tree Left Side View
42. Populating Next Right Pointers in Each Node
43. Populating Next Right Pointers in Each Node II
44. N-ary Tree Level Order Traversal
45. Cousins in Binary Tree
46. Check Completeness of a Binary Tree
47. Even Odd Tree

- 48. Reverse Odd Levels of Binary Tree

38.3 Strong Interview Practice

- 49. Vertical Order Traversal of a Binary Tree
- 50. Binary Tree Vertical Order Traversal
- 51. All Nodes Distance K in Binary Tree
- 52. Serialize and Deserialize Binary Tree
- 53. Binary Tree Right Side View
- 54. Maximum Width of Binary Tree
- 55. Find Bottom Left Tree Value

38.4 Hard / Stretch

- 56. Vertical Order Traversal of a Binary Tree
 - 57. Serialize and Deserialize Binary Tree
 - 58. Maximum Width of Binary Tree
 - 59. All Nodes Distance K in Binary Tree
 - 60. Amount of Time for Binary Tree to Be Infected
 - 61. Minimum Number of Operations to Sort a Binary Tree by Level
-

39. Suggested Study Order

Step 1 — Basic Level Order

Start with:

- 62. Binary Tree Level Order Traversal
- 63. Level Order Traversal II
- 64. Average of Levels

Goal:

Master queue and level size.

Step 2 — Depth and Level Statistics

Solve:

- 65. Maximum Depth BFS
- 66. Minimum Depth BFS
- 67. Largest Value in Each Row
- 68. Deepest Leaves Sum

Goal:

Use BFS for per-level computation.

Step 3 — Views and Ordering

Solve:

- 69. Right Side View
- 70. Left Side View
- 71. Zigzag Level Order

Goal:

Learn first/last and alternating-level logic.

Step 4 — Level Connections

Solve:

- 72. Populating Next Right Pointers I
- 73. Populating Next Right Pointers II

Goal:
Connect nodes within same level.

Step 5 — Structural Checks

Solve:

- 74. Cousins in Binary Tree
- 75. Check Completeness
- 76. Even Odd Tree

Goal:
Use BFS for structural validation.

Step 6 — Coordinate BFS

Solve:

- 77. Vertical Order Traversal
- 78. Maximum Width of Binary Tree

Goal:
Track row/column/index metadata.

40. Pattern Decision Table

level order	queue + level size	LC 102
bottom-up levels	level order + reverse	LC 107
zigzag	alternate fill index	LC 103
average per level	sum / level size	LC 637
maximum depth	count levels	LC 104
minimum depth	first leaf	LC 111
right side view	last node per level	LC 199
left side view	first node per level	variation
largest row value	max per level	LC 515
connect next pointers	prev pointer per level	LC 116/117
cousins	parent + level tracking	LC 993
complete tree	push nulls	LC 958
vertical traversal	row/col metadata + sort	LC 987
deepest leaves sum	overwrite sum per level	LC 1302
even odd rules	level parity	LC 1609

41. Final Cheat Sheet

Level Order

```
queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        TreeNode* node = q.front();
        q.pop();

        if (node->left != nullptr) {
            q.push(node->left);
        }

        if (node->right != nullptr) {
            q.push(node->right);
        }
    }
}
```

Right Side View

```
if (i == levelSize - 1) {
    result.push_back(node->val);
}
```

Left Side View

```
if (i == 0) {
    result.push_back(node->val);
}
```

Minimum Depth

```
if (node->left == nullptr && node->right == nullptr) {
    return depth;
}
```

Zigzag Index

```
int index = leftToRight ? i : levelSize - 1 - i;
level[index] = node->val;
```

Completeness Check

```
if (node == nullptr) {
    seenNull = true;
} else {
    if (seenNull) {
        return false;
    }

    q.push(node->left);
    q.push(node->right);
}
```

42. What to Master Before Chapter 22

Before moving to Binary Search Tree, make sure you can:

- Use queue for tree traversal
- Process levels using levelSize
- Return level order traversal
- Reverse bottom-up traversal
- Implement zigzag traversal
- Compute average per level
- Compute max/min depth with BFS
- Find right side view
- Find largest value in each row
- Connect next right pointers using BFS
- Check completeness of tree
- Track parent and level for cousins
- Understand when BFS is better than DFS
- Avoid mixing levels accidentally

43. Core Takeaway

Tree BFS is level-by-level traversal.

The main idea is:

queue stores nodes of upcoming levels
levelSize separates current level from next level

Use BFS when the problem talks about:

- levels
- rows
- minimum depth
- side views
- per-level statistics
- horizontal connections

- completeness

The most important line in Tree BFS is:

```
int levelSize = q.size();
```

That snapshot lets you process one level cleanly.

Tree BFS is usually not about complex recursion.

It is about disciplined queue processing.
